ECE PROJECT REPORT

AI-Based Component Placement Error Detection in PCB Assembly

Submitted by

VAISHNAVI D

Department of Electronics and Communication Engineering

Python / OpenCV prototype with implementation and testing guide

Project type	Computer vision / PCB image inspection
Platform	Google Colab on Android phone or computer
Tools	Python, OpenCV, NumPy, Matplotlib
Included	Report, source code, algorithm, test plan, viva questions

Academic honesty note: This document contains a runnable baseline prototype. Run it yourself and add your real screenshots and test observations before submitting. It does not invent experimental results or claim a model was trained.

Certificate / Declaration Template

I, **VAISHNAVI D**, declare that the report titled *"AI-Based Component Placement Error Detection in PCB Assembly"* is prepared for academic learning and demonstration. The code execution, screenshots, test observations and final conclusions must reflect the work actually performed.

Student signature: _____ Date: _____

Faculty guide: _____ Department: ECE

Use the official college format if supplied. Do not present an unsigned certificate as approved.

Abstract

Printed Circuit Board (PCB) assembly requires components to be placed in specified locations and orientations. Manual inspection can take time. This project presents a computer-vision prototype that compares a reference image of a correctly assembled PCB with a test image. The Python program uses OpenCV to resize images, convert them to grayscale, reduce small noise, calculate pixel differences, create a binary difference mask and highlight changed regions for human review. It can be run in Google Colab using an Android phone with internet access. The included baseline is image processing, not a trained AI classifier; lighting, reflections, camera angle and image misalignment may cause false alarms. Actual testing is required before making claims about accuracy. Future work can include image registration, labelled PCB defect data and a trained detection model.

Table of Contents

• 1. Introduction

• 2. Problem Statement, Aim and Objectives

• 3. Proposed Method

• 4. Requirements and Architecture

• 5. Algorithm

• 6. Source Code

• 7. Testing and Results

• 8. Limitations and Future Work

• 9. Conclusion

• 10. Viva Questions

• Appendix A. Running on Android

• Appendix B. References

1. Introduction

A Printed Circuit Board (PCB) mechanically supports and electrically connects electronic components. During assembly, a component may be missing, shifted, rotated or placed incorrectly. These defects can affect circuit operation. Automated Optical Inspection (AOI) systems use cameras and image-analysis methods to help inspect boards.

This student project demonstrates a small-scale inspection idea: compare a known-good reference image with a test image and highlight visual differences. It is a software prototype; it does not replace industrial inspection equipment.

2. Problem Statement, Aim and Objectives

Problem statement

Manual PCB inspection can be time-consuming. A simple image-comparison tool can help direct attention to regions that differ from a reference board.

Aim

To develop a beginner-friendly program that compares two PCB images and highlights regions that may need placement inspection.

Objectives

• Load a reference image and a test image.

• Resize the test image to match the reference dimensions.

• Convert both images to grayscale and smooth small noise.

• Calculate absolute pixel differences and threshold them.

• Find larger changed regions and draw boxes around them.

• Display the reference, difference mask and marked test image.

• Evaluate the prototype using controlled image pairs and record observations.

3. Proposed Method

Existing approach

Manual inspection and industrial AOI systems are used in PCB assembly. Industrial systems may use controlled lighting, calibrated cameras, board alignment, design data and trained models.

Proposed baseline

The prototype uses pixel-level image comparison. It is inexpensive and easy to explain, but it depends on similar image scale, orientation, lighting and camera position.

Difference calculation

For reference image R and test image T, the absolute difference is $D(x,y) = |R(x,y) - T(x,y)|$. A threshold τ creates a mask M: $M(x,y) = 255$ if $D(x,y) > \tau$; otherwise $M(x,y) = 0$. Connected changed regions are found and small regions are filtered. The code starts with $\tau = 30$ and a minimum contour area of 100 pixels. These are adjustable starting values, not universal settings.

AI note

The supplied baseline does not train a machine-learning model. Describe it honestly as an image-processing prototype. If your supervisor requires a genuine AI/ML model, a labelled dataset and trained, evaluated model must be added; do not claim this has already been completed.

4. Requirements and Architecture

Hardware

Android phone or computer with a browser and internet connection. No physical electronic hardware is required for the software demo.

Software

• Google Colab notebook environment

• Python runtime provided by Colab

• OpenCV for image processing

• NumPy for numerical operations

• Matplotlib for visualising output

System architecture

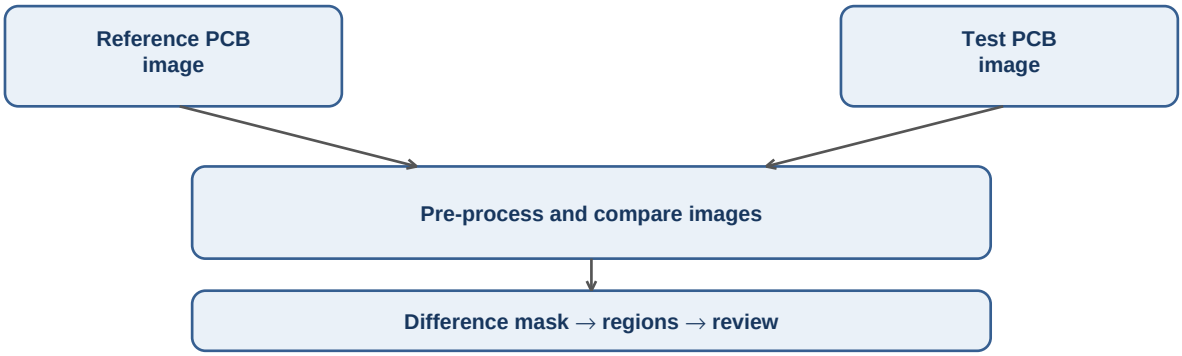

Figure 1. High-level architecture of the PCB image-comparison prototype.

Data flow

• Input: upload reference and test images.

• Pre-process: resize, grayscale and smooth.

• Compare: calculate absolute pixel differences.

• Filter: threshold the difference map and remove tiny noise.

• Locate: find contours and draw boxes around larger regions.

• Output: show suspected differences for human review.

5. Algorithm

```
START
  |
Upload reference PCB image
  |
Upload test PCB image
  |
Read images and check validity
  |
Resize test image to reference dimensions
  |
Convert to grayscale and smooth
  |
Calculate absolute pixel difference
  |
Threshold difference map
  |
Remove tiny noise and find contours
  |
Draw boxes around larger regions
  |
Display reference, mask and marked image
  |
END
```

Parameter table

Parameter	Value	Purpose
Gaussian blur	(5,5)	Reduces small pixel noise
Pixel threshold	30	Ignores smaller intensity changes
Minimum contour area	100 px	Filters tiny changed regions
Morphology kernel	3 × 3	Removes isolated pixels

6. Complete Starter Source Code

Paste the entire code below into one code cell in Google Colab. Upload one reference image when asked, then one test image. Use JPG or PNG files.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

print("Upload the CORRECT reference PCB image.")
ref_upload = files.upload()
if len(ref_upload) != 1:
    raise ValueError("Upload exactly one reference image.")

ref_name = next(iter(ref_upload))
reference = cv2.imread(ref_name)
if reference is None:
    raise ValueError("Could not read reference image. Use JPG or PNG.")

print("Now upload the PCB image to inspect.")
test_upload = files.upload()
if len(test_upload) != 1:
    raise ValueError("Upload exactly one test image.")

test_name = next(iter(test_upload))
test = cv2.imread(test_name)
if test is None:
    raise ValueError("Could not read test image. Use JPG or PNG.")

# Match dimensions; this does not fix rotation or perspective.
test = cv2.resize(test, (reference.shape[1], reference.shape[0]))

# Convert to grayscale and smooth minor noise.
ref_gray = cv2.cvtColor(reference, cv2.COLOR_BGR2GRAY)
test_gray = cv2.cvtColor(test, cv2.COLOR_BGR2GRAY)
ref_gray = cv2.GaussianBlur(ref_gray, (5, 5), 0)
test_gray = cv2.GaussianBlur(test_gray, (5, 5), 0)

# Calculate absolute pixel differences.
difference = cv2.absdiff(ref_gray, test_gray)
_, mask = cv2.threshold(difference, 30, 255, cv2.THRESH_BINARY)

# Remove tiny isolated pixels.
kernel = np.ones((3, 3), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)

# Find connected changed regions.
contours, _ = cv2.findContours(
    mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
)
result = test.copy()
```

```
regions = 0

for contour in contours:
    if cv2.contourArea(contour) > 100:
        x, y, w, h = cv2.boundingRect(contour)
        cv2.rectangle(result, (x, y), (x+w, y+h), (0, 0, 255), 2)
        regions += 1

if regions == 0:
    print("PASS: No significant image differences detected.")
else:
    print("CHECK: Possible differences need human inspection.")
    print("Highlighted regions:", regions)

plt.figure(figsize=(14, 5))
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(reference, cv2.COLOR_BGR2RGB))
plt.title("Reference PCB")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(mask, cmap="gray")
plt.title("Difference mask")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Regions to inspect")
plt.axis("off")
plt.tight_layout()
plt.show()
```

Interpretation: PASS means no changed region exceeded the thresholds; it does not guarantee the PCB is correct. CHECK means differences were found; it does not prove a placement defect. Review the image manually.

7. Testing Plan and Results Record

Run the tests yourself and fill in the actual-results column. The expected observations below are not claimed experimental results.

ID	Test input	Expected observation	Actual result
T1	Same image twice	Ideally no significant difference regions	Fill after running
T2	Similar board images and lighting	Some small differences may appear	Fill after running
T3	Visible change in a component/area	Changed area may be highlighted	Fill after running
T4	Different brightness or shadow	False alarms may occur	Fill after running
T5	Different angle/alignment	Many regions may be highlighted	Fill after running

Evidence to add after running

• Screenshot of code running in Colab.

• Reference PCB and test PCB images.

• Screenshot of the difference mask and highlighted output.

• Completed testing table with actual observations.

• Short explanation of failures and limitations.

Evaluation

For a real accuracy evaluation, use labelled images and record true detections, missed defects and false alarms. Do not report a percentage accuracy unless it is calculated from a defined labelled test set. This baseline does not produce validated classification accuracy by itself.

8. Limitations and Future Work

Limitations

• Sensitive to lighting, shadows, reflections, scale and camera angle.

• Resizing matches dimensions but does not align rotated or perspective-distorted images.

• May flag harmless changes or miss subtle placement defects.

• Does not identify component types or check PCB design rules.

• Not suitable for production quality control without extensive validation.

Future work

• Use controlled lighting and fixed camera placement.

• Align images using feature matching or fiducial markers.

• Collect labelled images of correct and defective PCB assemblies.

• Train and evaluate an object-detection/classification model.

• Compare component locations against PCB design coordinates.

• Create a user interface and export inspection reports.

Safety

The software demo does not require live electronics. If using a physical PCB, power it off before handling and avoid shorting pins or touching exposed powered circuits.

9. Conclusion

A low-cost PCB image-inspection prototype has been specified using Python and OpenCV. It compares a reference image with a test image, creates a difference mask and highlights larger changed regions. The program can be run in Google Colab from an Android phone. Its performance must be established through actual tests, and it must not be described as a trained AI defect classifier until a model is trained and validated.

10. Viva Questions and Answers

1. What is a PCB?
A Printed Circuit Board supports and electrically connects components.

2. What is the project aim?
Compare PCB images and highlight areas needing inspection.

3. What is OpenCV?
A library for computer vision and image processing.

4. Why is a reference image needed?
It represents the expected appearance of a correctly assembled board.

5. What is absolute difference?
The absolute pixel-intensity difference between corresponding image locations.

6. What is thresholding?
Separating pixel differences above a selected value.

7. What are contours?
Boundaries of connected regions in a binary image.

8. Is this a trained AI model?
No. The baseline uses image-processing rules; a trained model is future work.

9. Why can false alarms happen?
Lighting, shadows, reflections, camera angle and misalignment.

10. How can it be improved?
Align images, use labelled data, train a model and test it on unseen images.

11. Why use Google Colab?
It runs Python in a browser without local setup.

12. What does PASS mean?
No region exceeded the thresholds; it does not guarantee a defect-free PCB.

Appendix A. Run on an Android Phone

• Open Chrome and visit https://colab.research.google.com/.

• Sign in and select New notebook.

• Run a small code cell: print('Project setup successful').

• Add a new code cell and paste the full code from Section 6.

• Run it and upload one reference image when prompted.

• Upload one test image when prompted.

• Wait for the three plots, then save screenshots.

• Repeat the tests and complete the actual-results column.

Troubleshooting

Colab will not load: Check internet, refresh, or open the link in Chrome.

Image cannot be read: Use a valid JPG or PNG file.

Too many boxes: Images may differ in lighting or alignment; use more consistent images.

No box on changed image: The change may be small or below the threshold; inspect the difference mask.

Appendix B. References

• OpenCV Documentation: https://docs.opencv.org/

• Google Colab: https://colab.research.google.com/

• NumPy Documentation: https://numpy.org/doc/

• Matplotlib Documentation: https://matplotlib.org/stable/

• Szeliski, R., Computer Vision: Algorithms and Applications, 2nd edition, Springer, 2022.